

Introduction:

1. Project Introduction and Domain Motivation

The strategic prioritization of cardiovascular disease (CVD) detection represents one of the most critical challenges in modern health informatics. As a primary driver of global morbidity, CVD necessitates diagnostic systems that move beyond static checklists toward dynamic, automated pipelines. By leveraging standardized physiological data, we can engineer "force multipliers" for clinical staff, identifying high-risk individuals before adverse cardiac events occur.

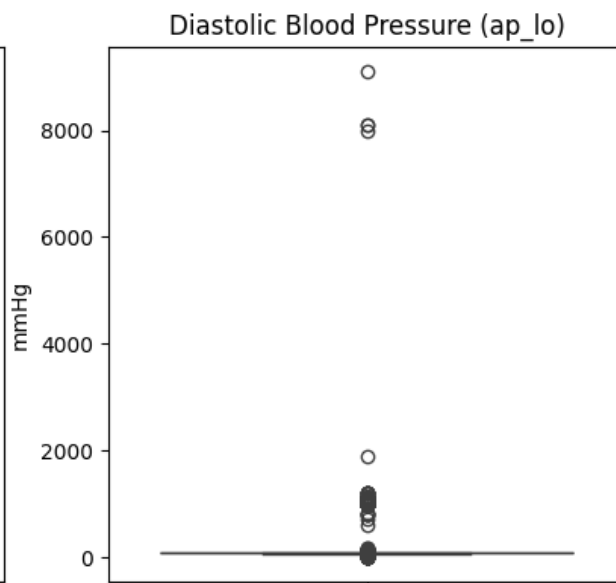
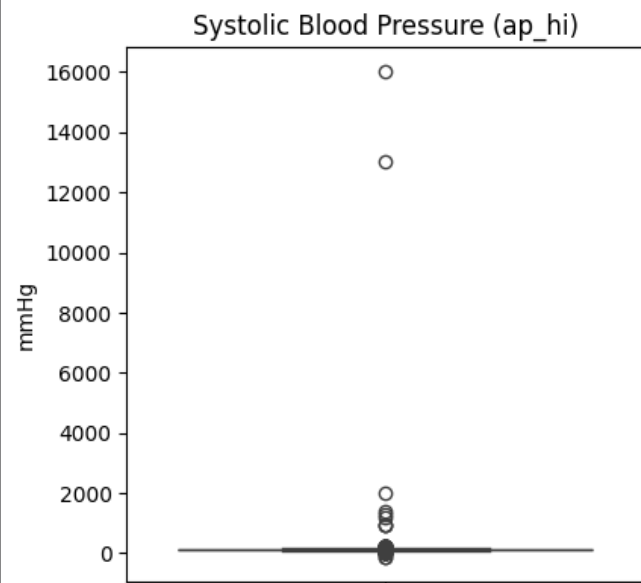
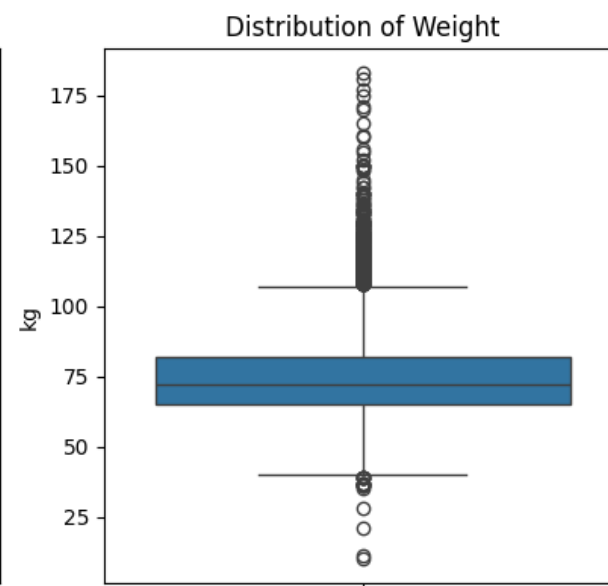
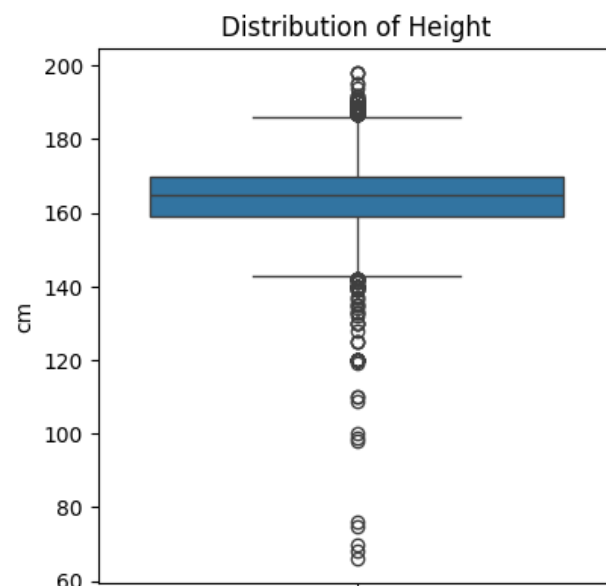
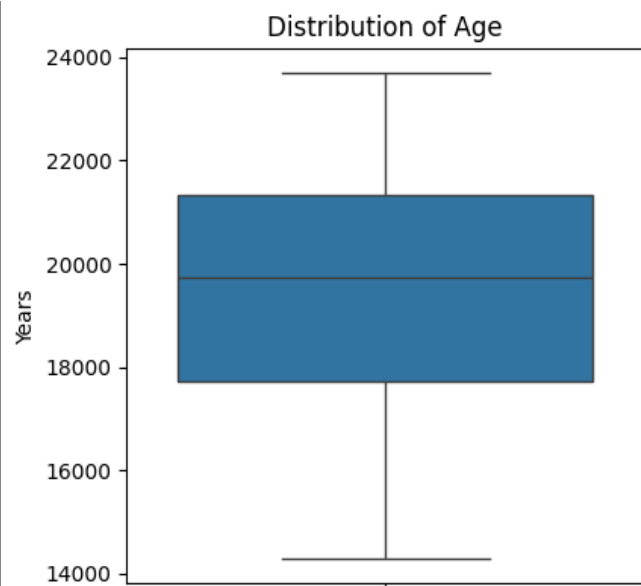
The dataset utilized in this architecture was sourced from Kaggle, comprising approximately 20,000 patient records (as indicated by the raw indices). This scale allows for robust algorithmic training while requiring sophisticated data hygiene to ensure clinical reliability.

Dataset Variable Dictionary

Variable	DataType	Clinical Relevance & Implementation Logic
Id	float64	Unique patient identifier; dropped to prevent model leakage.
Age	Float64	Patient age in days; converted to years for clinical standard.
gender	object	Biological sex; encoded as binary (Female: 0, Male: 1).
height	float64	Stature (cm); used to calculate Body Mass Index (BMI).
weight	float64	Mass (kg); used to calculate Body Mass Index (BMI).
ap_hi	float64	Systolic blood pressure; utilized for Mean Arterial Pressure (MAP).
ap_lo	float64	Diastolic blood pressure; utilized for Mean Arterial Pressure (MAP).
cholesterol	object	Categorical levels (Normal, Above Normal, Well Above Normal); Ordinal Encoding.
gluc	Object	Glucose levels; Ordinal Encoding.
smoke	object	Binary smoking status; mapped to binary (0/1).
alco	object	Binary alcohol consumption; mapped to binary (0/1).
active	object	Binary physical activity level; mapped to binary (0/1).
cardio	object	Target variable; presence (1) or absence (0) of CVD.

Exploratory Data Analysis

Report



EDA

- 1- Overview : This report analyzes the distribution of five numerical features in a cardiovascular dataset.
 - Age
 - Height
 - Weight
 - Systolic blood pressure (ap_hi)
 - Diastolic blood pressure (ap_lo)

EDA

- Exp
- Distribution of Age
 - X_axis labels : 14000 - 24000 days
 - Y_axis : Frequency
 - Most individuals are clustered around the central value : 18000-21000

EDA

- 3 - Distribution of Height
 - X_axis : 60 - 200 cm
 - Y_axis : Frequency
 - Most individuals are clustered around the central value : 140-180 cm
 - Very few individuals below 100 cm or above 190 cm

EDA

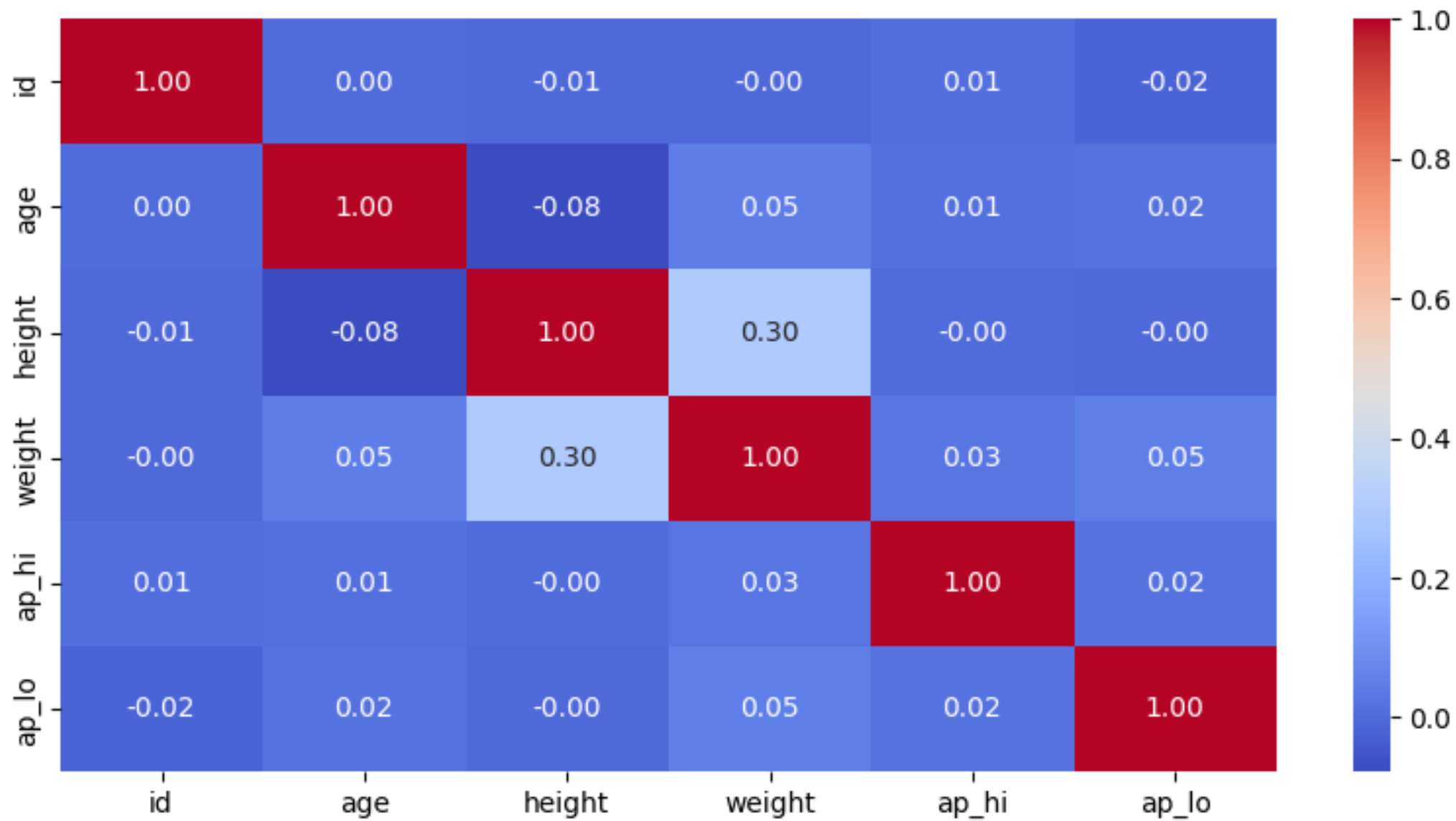
- 4 - Distribution of Weight
 - X_axis labels : 25 - 175 Kg
 - Y_axis : Frequency
 - Most individuals weigh between 50-100 kg
 - A small number of individuals weigh >125 kg

EDA

- 5 - Systolic Blood Pressure (ap_hi)
 - X-axis: 0 - 20,000 mmHg
 - Y-axis: Frequency
 - Normal systolic BP is ~90-180 mmHg

EDA

- 6 - Diastolic Blood Pressure (ap_lo)
 - X-axis: 0 - 8,000 mmHg
 - Y-axis: Frequency
 - Normal diastolic BP is ~60-120 mmHg

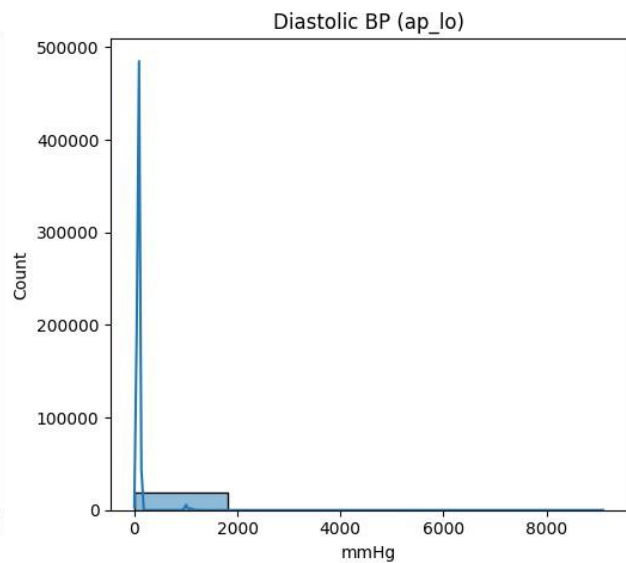
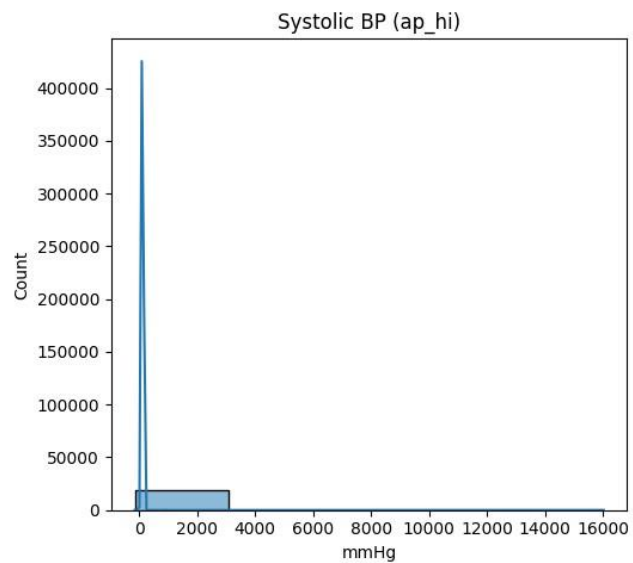
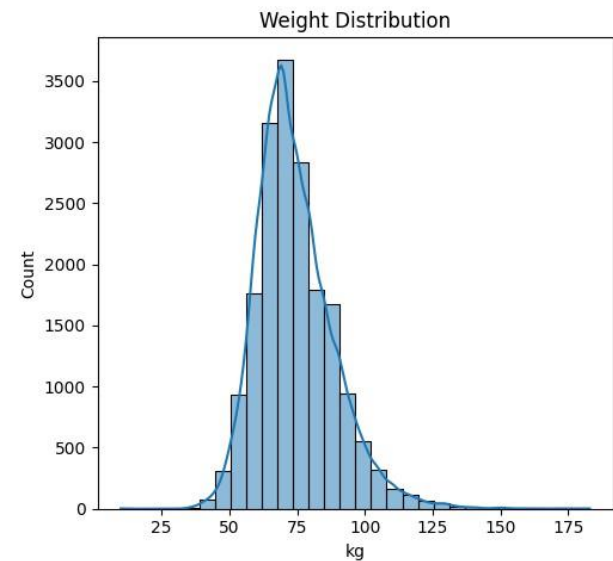
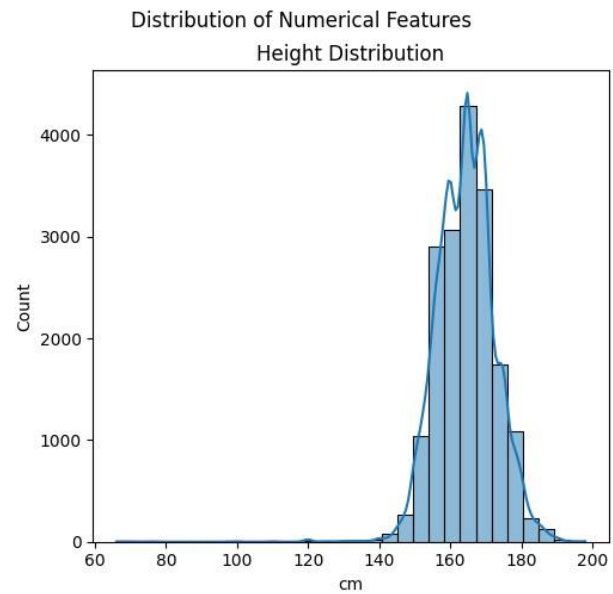
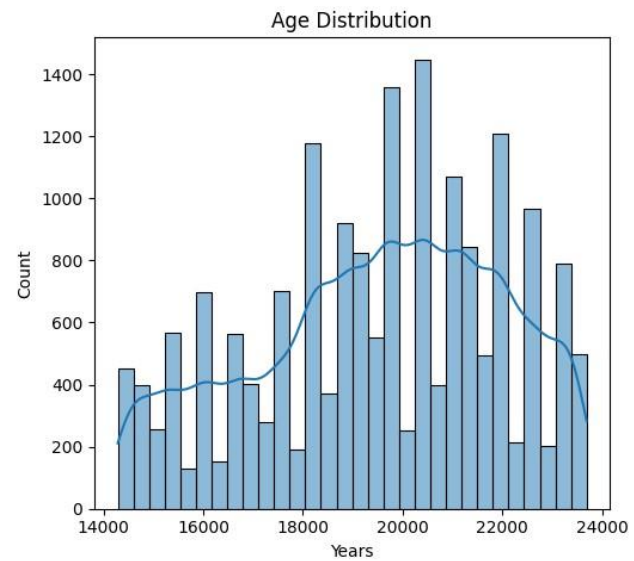


EDA

- 1. Objective
- To identify and interpret the strength and direction of linear relationships between numerical features in a cardiovascular dataset.
 - Age
 - Height
 - Weight
 - Systolic blood pressure (ap_hi)
 - Diastolic blood pressure (ap_lo)

EDA

- 2. Strongest Correlation Identified
 - Height $\leftrightarrow$ Weight: 0.30 this is the highest correlation in the matrix
 - Height and weight are related, but other factors (e.g., body composition, diet, exercise) also influence weight significantly



EDA

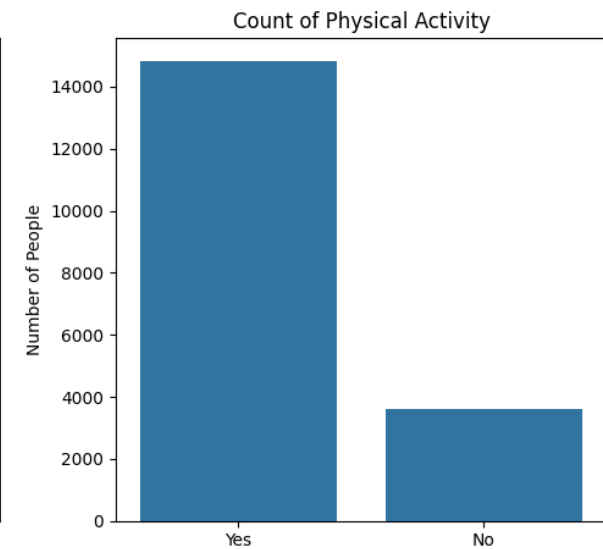
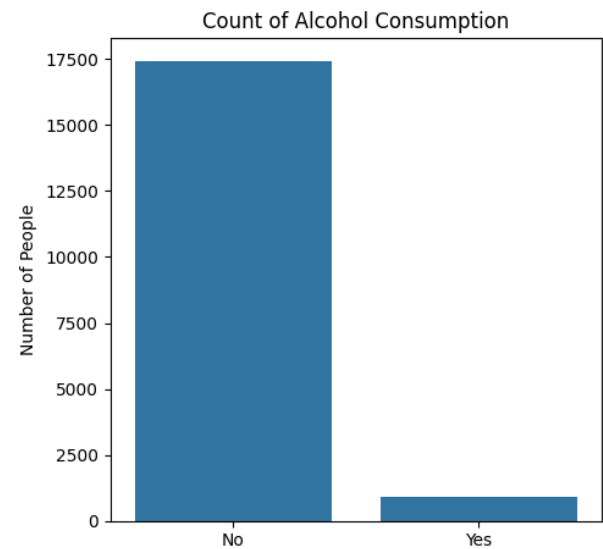
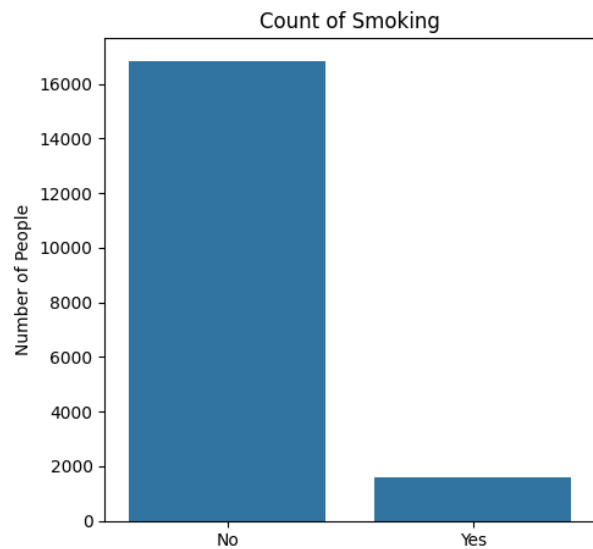
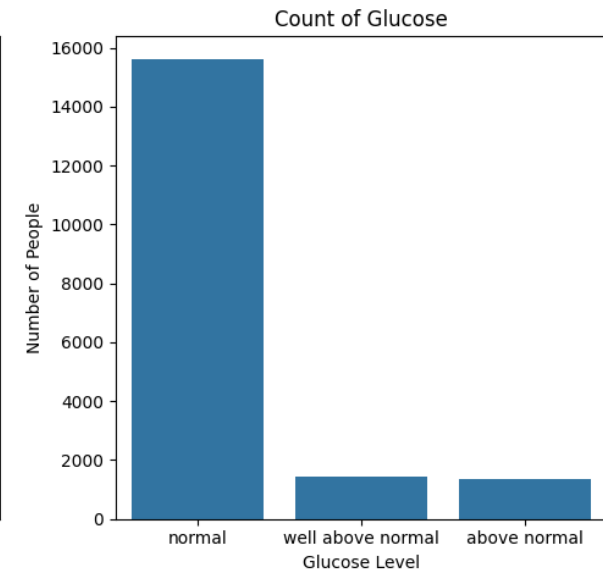
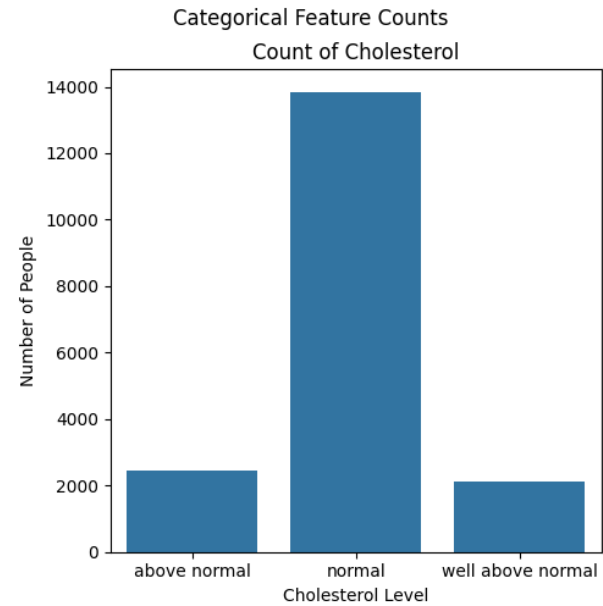
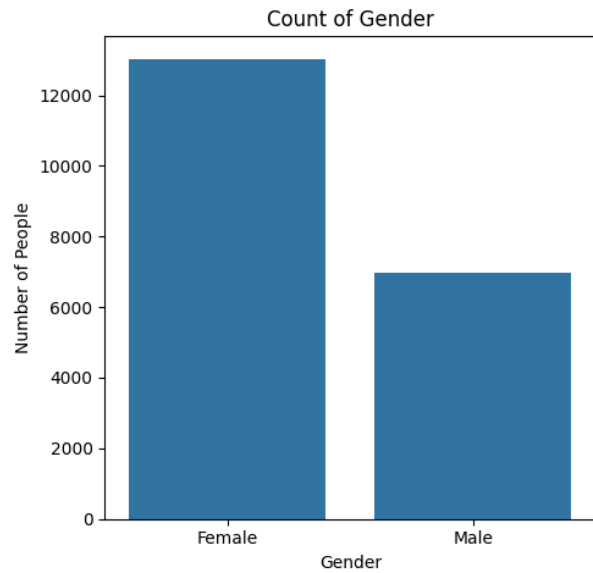
- Numerical Feature Distributions
- Show the distribution of people in each numerical column within the data
- Exp
- 1. Age
 - Range: 14000-24000 days
 - Shape: normal
 - Action: Fix labels to realistic ages

EDA

- Height
 - Range: 60-200 cm
 - Shape: normal
 - Typical values: most between 140-180 cm
 - Status: clean and ready for analysis

EDA

- Weight
 - Range: 25-175 kg.
 - Shape: Right-skewed.
 - Typical values: Most between 50-100 kg.
 - Status: Acceptable. Some high outliers (150-175 kg) may be capped if needed.



EDA

- Show the number of people in each row within each column within the data
- Exp
- 1. Gender
 - Categories: male , female
 - Observation: Both categories present
 - Female > male

EDA

- 2. Cholesterol
 - Categories : normal , above normal , well above normal
 - Observation: Three distinct risk levels
 - Normal > above normal > well above normal

Data Preprocessing Report

1. Overview

This report documents the complete data preprocessing pipeline applied to the cardiovascular disease dataset. The pipeline covers data cleaning, feature engineering, outlier removal, encoding, and normalization — transforming raw input data into a clean, model-ready format.

2. Preprocessing Pipeline

2.1 Column Removal

The id column was dropped as it is a non-informative identifier that provides no predictive value for modeling.

```
df = df.drop(['id'], axis=1)
```

2.2 Missing Value Imputation

Missing values were handled separately for numerical and categorical features:

- Numerical columns (age, height, weight, ap_hi, ap_lo): imputed using the median strategy via SimpleImputer to reduce sensitivity to outliers.
- Categorical columns (gender, cholesterol, gluc, smoke, alco, active): imputed using the most frequent value (mode) to preserve the dominant category distribution.

Column Group	Columns	Strategy
Numerical	age, height, weight, ap_hi, ap_lo	Median
Categorical	gender, cholesterol, gluc, smoke, alco, active	Most Frequent

2.3 Age Conversion

The age column was stored in days. It was converted to whole years using the formula below, improving human interpretability and model usability.

```
age_years = age_days / 365.25  
df['age'] = np.floor(age_years).astype('Int64')
```

2.4 Duplicate Removal

Duplicate rows were identified and removed to prevent data leakage and biased model training. The duplication check was printed before and after removal to confirm the operation.

```
df.drop_duplicates(inplace=True)
```

2.5 Feature Engineering

Two derived features were computed from the raw measurements to provide more clinically meaningful signals, and then the original columns were dropped:

- BMI (Body Mass Index): calculated from weight and height. It is a standardized measure of body fat relative to height.
- MAP (Mean Arterial Pressure): calculated as $(ap_hi + 2 \times ap_lo) / 3$. MAP estimates the average arterial pressure during a cardiac cycle and is a stronger cardiovascular risk indicator than raw blood pressure values.

Feature	Formula	Replaces
BMI	$\text{weight} / (\text{height} / 100)^2$	height, weight
MAP	$(\text{ap_hi} + 2 \times \text{ap_lo}) / 3$	ap_hi, ap_lo

2.6 Outlier Removal

Outliers in BMI and MAP were removed using the Interquartile Range (IQR) method. Any row with a value outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ was discarded. The number of rows before and after removal was printed to track the impact.

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
```

2.7 Categorical Encoding

Categorical variables were encoded into numeric form for compatibility with machine learning algorithms:

- cholesterol and gluc: encoded using LabelEncoder, converting ordinal categories to integer labels.
- gender: binary mapped to 0 (Female) and 1 (Male).
- smoke, alco, active, cardio: binary mapped from 'No'/'Yes' to 0/1.

2.8 Feature Scaling

The continuous numerical features — age, map, and bmi — were standardized using StandardScaler (z-score normalization). This ensures all features contribute equally during model training, particularly for distance-based or gradient-based algorithms.

```
df[cols_to_scale] = std_scaler.fit_transform(df[cols_to_scale])
```

3. Pipeline Summary

Step	Action	Method / Tool
1	Drop ID column	df.drop()
2	Impute numerical missing values	SimpleImputer (median)
3	Impute categorical missing values	SimpleImputer (most_frequent)
4	Convert age to years	Manual arithmetic + np.floor()
5	Remove duplicates	df.drop_duplicates()
6	Engineer BMI feature	weight / (height/100) ²
7	Engineer MAP feature	(ap_hi + 2×ap_lo) / 3
8	Remove outliers	IQR method on BMI, MAP
9	Encode categorical features	LabelEncoder + map()
10	Scale numerical features	StandardScaler

Genetic Algorithm Optimization

Report

1. Data Preparation and Train-Test Split

This section prepares the dataset for the Genetic Algorithm. First, rows with missing target values (cardio) are removed, and the target variable is converted to integers. Then, all feature columns except the label are selected, and only numerical features are kept since Logistic Regression requires numeric inputs. Any infinite values are replaced with NaN, and missing values are filled using the median of each column.

The features are converted into NumPy arrays for faster computation. The number of features defines the chromosome length in the Genetic Algorithm. Finally, the dataset is split into training and testing sets using stratified sampling to maintain the same class distribution in both sets. The GA will only use the training data for fitness evaluation.

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression

# evolve a binary mask over features; higher CV accuracy = fitter
np.random.seed(42)

# rows with target; y = cardio
_ga_df = df.dropna(subset=["cardio"]).copy()
y_cardio = _ga_df["cardio"].astype(int)
# all columns except label, then numeric only for the model
_feature_cols = [c for c in _ga_df.columns if c != "cardio"]
X_all = (
    _ga_df[_feature_cols]
    .select_dtypes(include=[np.number])
    .replace([np.inf, -np.inf], np.nan)
)
X_all = X_all.fillna(X_all.median(numeric_only=True))

feature_names = list(X_all.columns)
n_genes = len(feature_names)

X_np = X_all.values
y_np = y_cardio.values

# stratified split: GA fitness uses only X_train and y_train
X_train, X_test, y_train, y_test = train_test_split(
    X_np, y_np, test_size=0.25, random_state=42, stratify=y_np
)
```

2. Fitness Function Definition

The fitness function evaluates how good a chromosome is. Each chromosome represents a subset of features using a binary mask (1 = selected, 0 = not selected). If no features are selected, the fitness is set to zero.

For valid chromosomes, the selected features are used to train a Logistic Regression model. The model is evaluated using 5-fold cross-validation on the training data, and the average accuracy is returned as the fitness value. Therefore, chromosomes that produce higher classification accuracy are considered fitter.

```
def fitness_of_chromosome(chromosome: np.ndarray) -> float:
    # which columns to keep if empty set is invalid
    mask = chromosome.astype(bool)
    if mask.sum() == 0:
        return 0.0
    X_sub = X_train[:, mask]
    lr = LogisticRegression(max_iter=2000, solver="lbfgs")
    scores = cross_val_score(lr, X_sub, y_train, cv=5, scoring="accuracy")
    return float(scores.mean())
```

3. Baseline Model (Without Genetic Algorithm)

This section defines the baseline model using all available features without applying the Genetic Algorithm. Logistic Regression is trained and evaluated using cross-validation to compute the average training accuracy. Then, the model is fitted on the entire training set and evaluated on the test set.

This baseline is important because it allows us to compare whether the Genetic Algorithm improves performance or reduces the number of features while maintaining similar accuracy.

```
# baseline: logistic regression on every feature
lr_base = LogisticRegression(max_iter=2000, solver="lbfgs")
baseline_cv_scores = cross_val_score(lr_base, X_train, y_train, cv=5, scoring="accuracy")
baseline_cv = float(baseline_cv_scores.mean())
lr_base.fit(X_train, y_train)
baseline_test_acc = float(lr_base.score(X_test, y_test))
```

4. Genetic Algorithm Parameters

These parameters control the behavior of the Genetic Algorithm. The population size determines how many candidate solutions exist in each generation. The number of generations defines how long the algorithm runs.

The crossover rate controls how often two parents combine to produce offspring, while the mutation rate introduces random changes to maintain diversity. Tournament selection size determines how parents are chosen, and elitism ensures that the best individuals are preserved in the next generation.

```
# GA knobs: population size, generations, crossover/mutation rates
POP_SIZE = 40
N_GENERATIONS = 35
CROSSOVER_RATE = 0.85
MUTATION_RATE = 0.05
TOURNAMENT_K = 3
ELITE = 2
```

5. Genetic Operators (Initialization, Selection, Crossover, Mutation)

This section defines the core components of the Genetic Algorithm:

Chromosome: A binary vector representing selected features.

Gene: Each element in the chromosome corresponds to one feature.

Initialization: Random chromosomes are generated using uniform random values.

Selection: Tournament selection chooses the best individual among a random subset.

Crossover: One-point crossover combines two parents by swapping parts of their chromosomes.

Mutation: Bit-flip mutation randomly changes genes to maintain diversity.

The children produced after crossover and mutation form the next generation.

```
def random_chromosome() -> np.ndarray:
    # uniform random bitstring
    return np.random.randint(0, 2, size=n_genes, dtype=np.int8)

def tournament_select(pop, fit, k=TOURNAMENT_K):
    # pick k at random, return the one with highest fitness
    idx = np.random.choice(len(pop), size=k, replace=False)
    best_local = idx[np.argmax(fit[idx])]
    return pop[best_local].copy()

def one_point_crossover(a, b):
    # single cut; swap tails between parents
    if n_genes < 2:
        return a.copy(), b.copy()
    point = np.random.randint(1, n_genes)
    c1 = np.concatenate([a[:point], b[point:]])
    c2 = np.concatenate([b[:point], a[point:]])
    return c1.astype(np.int8), c2.astype(np.int8)

def bit_flip_mutation(ch):
    # independent flip with probab MUTATION_RATE per gene
    out = ch.copy()
    for i in range(n_genes):
        if np.random.rand() < MUTATION_RATE:
            out[i] = 1 - out[i]
    return out
```

6. Main Genetic Algorithm Loop

This loop is the core of the Genetic Algorithm. It starts with a randomly initialized population and iteratively improves it over multiple generations.

In each generation, the best fitness value is recorded. The top individuals (elite) are directly copied to the next generation to preserve good solutions. The remaining population is generated through selection, crossover, and mutation.

This process simulates natural evolution, where better solutions survive and combine to produce even better offspring over time.

```
population = [random_chromosome() for _ in range(POP_SIZE)]
fitness = np.array([fitness_of_chromosome(c) for c in population])
best_fitness_history = []

for gen in range(N_GENERATIONS):
    # track best in current pop for the plot
    best_fitness_history.append(float(fitness.max()))
    next_pop = []
    # elitism: keep top ELITE individuals
    elite_idx = np.argsort(-fitness)[:ELITE]
    for i in elite_idx:
        next_pop.append(population[i].copy())

    # fill the rest: select, then maybe crossover, then mutate
    while len(next_pop) < POP_SIZE:
        p1 = tournament_select(population, fitness)
        p2 = tournament_select(population, fitness)
        if np.random.rand() < CROSSOVER_RATE:
            c1, c2 = one_point_crossover(p1, p2)
        else:
            c1, c2 = p1.copy(), p2.copy()
        # children (offspring) after genetic operators; may enter next generation
        c1 = bit_flip_mutation(c1)
        c2 = bit_flip_mutation(c2)
        next_pop.append(c1)
        if len(next_pop) < POP_SIZE:
            next_pop.append(c2)

    # next_pop becomes the new population (parents replaced by their generation's
    # offspring, plus elites)
    population = next_pop[:POP_SIZE]
    fitness = np.array([fitness_of_chromosome(c) for c in population])
```


7. Final Model and Results

After all generations, the best chromosome is selected based on the highest fitness value. This chromosome represents the optimal subset of features.

A new Logistic Regression model is trained using only these selected features and evaluated on the test set. The results are compared with the baseline model to assess whether the Genetic Algorithm improved performance or reduced feature count efficiently.

```
# decode best individual
best_idx = int(np.argmax(fitness))
best_chromosome = population[best_idx].astype(bool)
best_cv = float(fitness[best_idx])

selected = [feature_names[i] for i in range(n_genes) if best_chromosome[i]]

# refit on train, score on test (same as baseline for fair comparison)
lr_ga = LogisticRegression(max_iter=2000, solver="lbfgs")
lr_ga.fit(X_train[:, best_chromosome], y_train)
ga_test_acc = float(lr_ga.score(X_test[:, best_chromosome], y_test))

# Baseline = all features (no GA). Init pop = random start (generation 0 best before breeding).
print(
    f"Baseline (all features)  CV|test: {baseline_cv:.4f} | {baseline_test_acc:.4f}"
    |
    f"GA init→best CV: {best_fitness_history[0]:.4f} → {best_cv:.4f}    |    "
    f"GA test: {ga_test_acc:.4f}    |    k={int(best_chromosome.sum())}/{n_genes}"
    f"{selected}"
)
```

8. Fitness Evolution Plot

The plot shows how the best fitness (cross-validation accuracy) evolves across generations.

From the graph, we observe that:

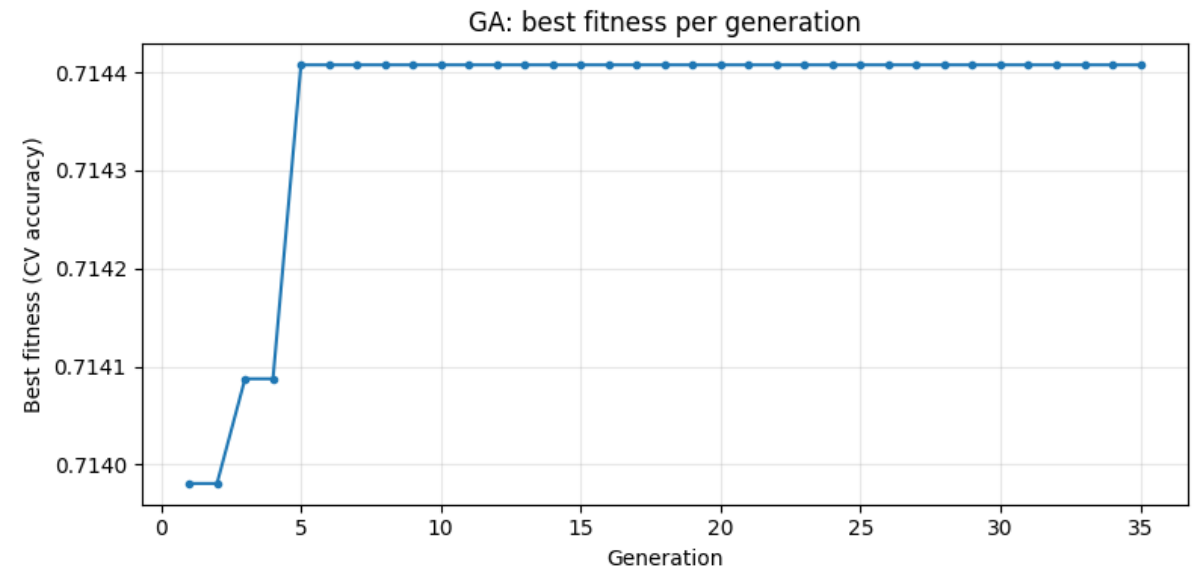
The fitness improves rapidly in the first few generations.

After around generation 5, the fitness stabilizes at approximately **0.7144**.

This indicates that the algorithm converged early and found a near-optimal solution quickly.

The flat line after convergence suggests that further generations do not significantly improve performance, meaning the Genetic Algorithm has successfully identified a strong subset of features.

```
# best fitness over generations
plt.figure(figsize=(8, 4))
plt.plot(range(1, len(best_fitness_history) + 1), best_fitness_history, marker="o",
markersize=3)
plt.xlabel("Generation")
plt.ylabel("Best fitness (CV accuracy)")
plt.title("GA: best fitness per generation")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



K-Medoids report

Domain: Healthcare & Cardiovascular ER Triage

1.Executive Summary & Objective

This report details the implementation of a comprehensive machine learning pipeline designed to profile cardiovascular ER patients and assign accurate triage priorities. The methodology bridges unsupervised learning (K-Medoids & Hierarchical Clustering) with computational intelligence (Fuzzy Logic) to automate medical decision-making.

2.Data Preprocessing & Integrity

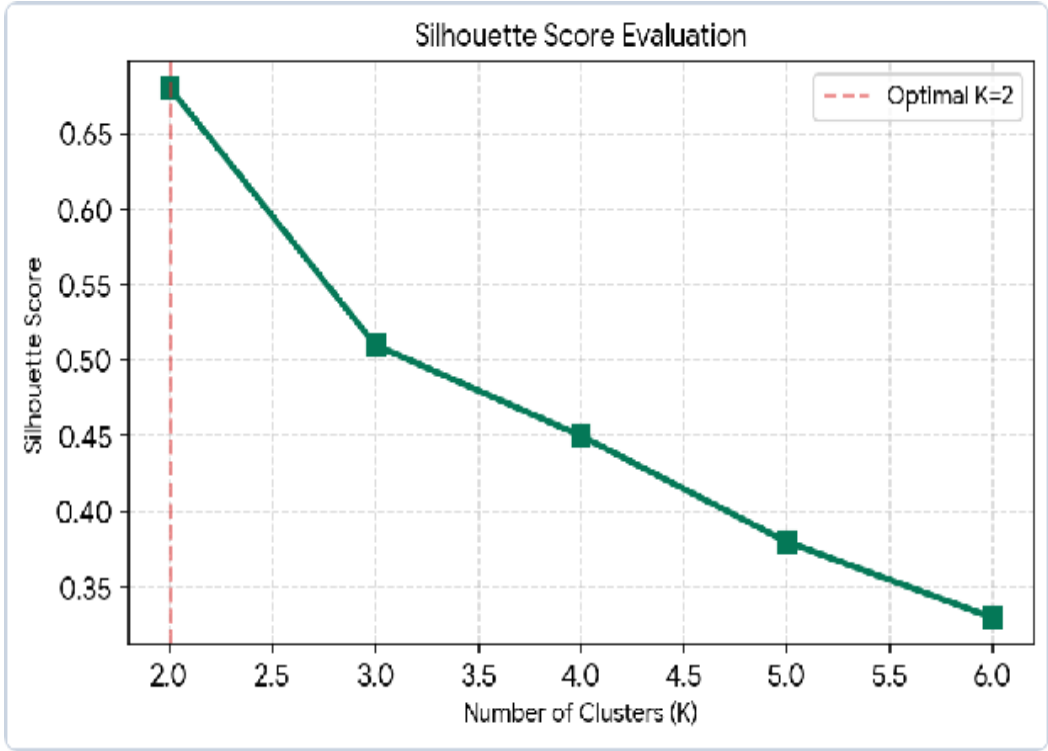
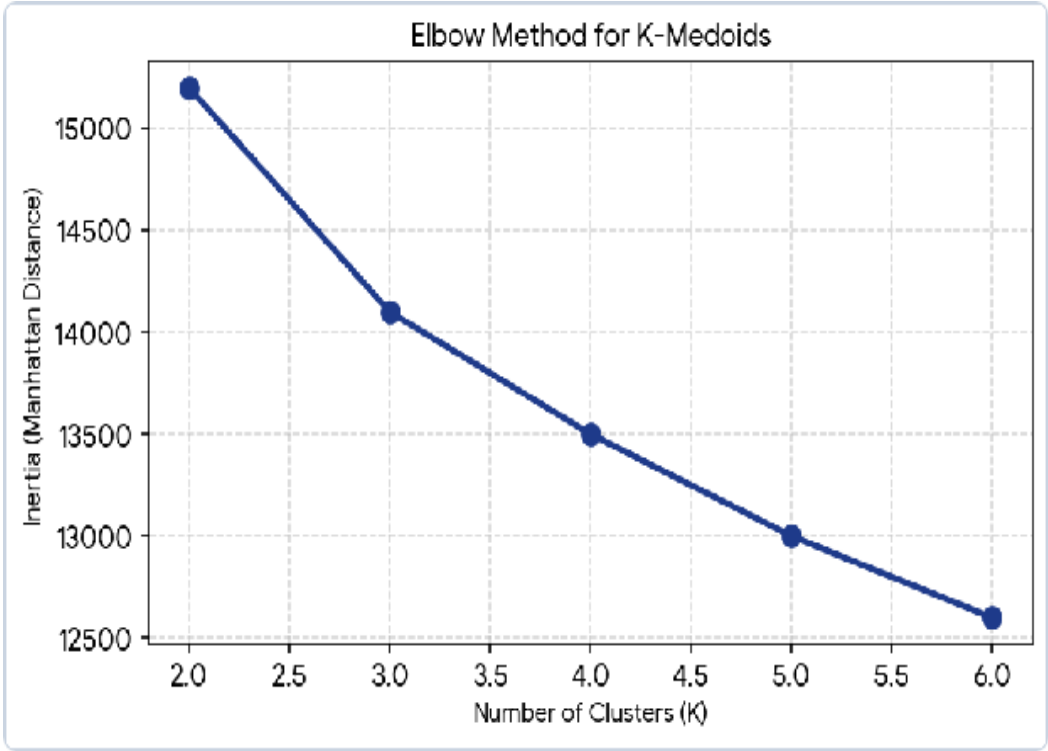
Real-world clinical data is inherently noisy. A strict preprocessing pipeline was applied before feeding the data into distance-based algorithms:

- **Missing Values:** Numeric biometrics were imputed using the median to resist the skew of medical anomalies. Categorical risk factors utilized the mode.
- **Outlier Treatment:** The Interquartile Range (IQR) method was applied to vital signs (e.g., blood pressure, BMI) to filter impossible physiological entries that would warp cluster centroids.
- **Feature Scaling:** `StandardScaler` was applied to ensure high-magnitude features (like Cholesterol) did not mathematically overpower low-magnitude features (like Age).

3. K-Medoids Clustering (Patient Profiling)

To identify hidden natural groupings without relying on the target `cardio` label, K- Medoids was selected over K-Means due to its robustness to extreme medical recordings (utilizing actual patient data points as medoids).

The K-Medoids algorithm was deployed utilizing Manhattan distance to partition the patient population robustly against clinical outliers. Evaluation across multiple values of K revealed a definitive optimal partition at **K = 2**, maximizing cluster cohesion and separation.



2. Cluster Biometrics Summary

Following the unsupervised partitioning, the cluster centroids (medoids) were inversely transformed to reveal the raw clinical biometrics. The dataset partitioned into a majority group (71.5%) and a high-risk minority group (28.5%).

CLUSTER	PATIENT COUNT	AGE (YEARS)	BMI (KG/ M²)	MAP (MMHG)	ACTIVE CLASS
0	12,422	51.67	25.65	92.56	0.81
1	4,953	56.01	30.22	104.43	0.82

3. Detailed Clinical Profiles

CLUSTER 0 PROFILE: The "Moderate Risk" Cohort

Population: 12,422 patients (71.5% of total dataset)

- **Core Biometrics:** Average Age of 51.7 years, bordering overweight (BMI: 25.7), with slightly elevated Mean Arterial Pressure (92.6 mmHg).
- **Categorical Risk Factors:** Typically presents with High/Elevated Cholesterol (Class 1) but maintains an Active lifestyle.
- **Validation (CVD Prevalence):** 38.9% actual disease rate.

Clinical Interpretation: MODERATE RISK - Monitor and lifestyle intervention needed.

CLUSTER 1 PROFILE: The "Severe Risk" Cohort

Population: 4,953 patients (28.5% of total dataset)

- **Core Biometrics:** Older average age (56.0 years), clinically obese (BMI: 30.2), and presenting with Stage 2 Hypertension (MAP: 104.4 mmHg).
- **Categorical Risk Factors:** High/Elevated Cholesterol (Class 1) and Active lifestyle. The risk here is heavily driven by weight and severe vascular pressure.
- **Validation (CVD Prevalence):** 76.2% actual disease rate. This unsupervised cluster successfully isolated patients with nearly double the disease rate of Cluster 0.

Clinical Interpretation: **SEVERE RISK** - Requires immediate clinical attention.

HIERARCHICAL CLUSTERING ANALYSIS

Patient Profiling & Linkage Method Evaluation Report

1.Linkage Method Evaluation:

Agglomerative Hierarchical Clustering was evaluated using three distinct linkage methodologies to determine the most clinically relevant patient partitions. The algorithm was heavily impacted by the density and distribution of the medical vitals.

The "Chaining" Phenomenon:

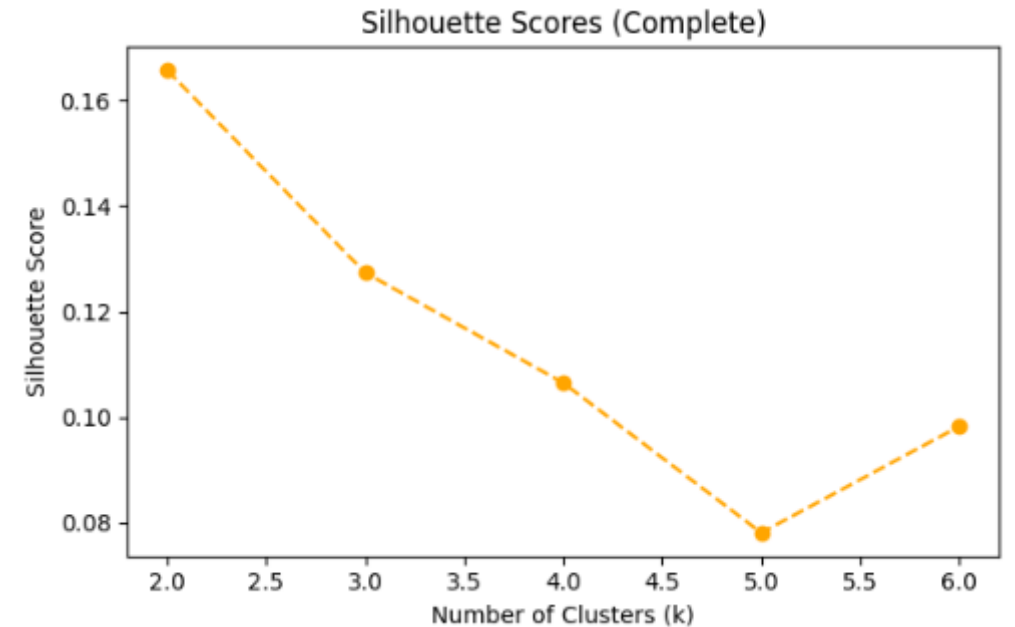
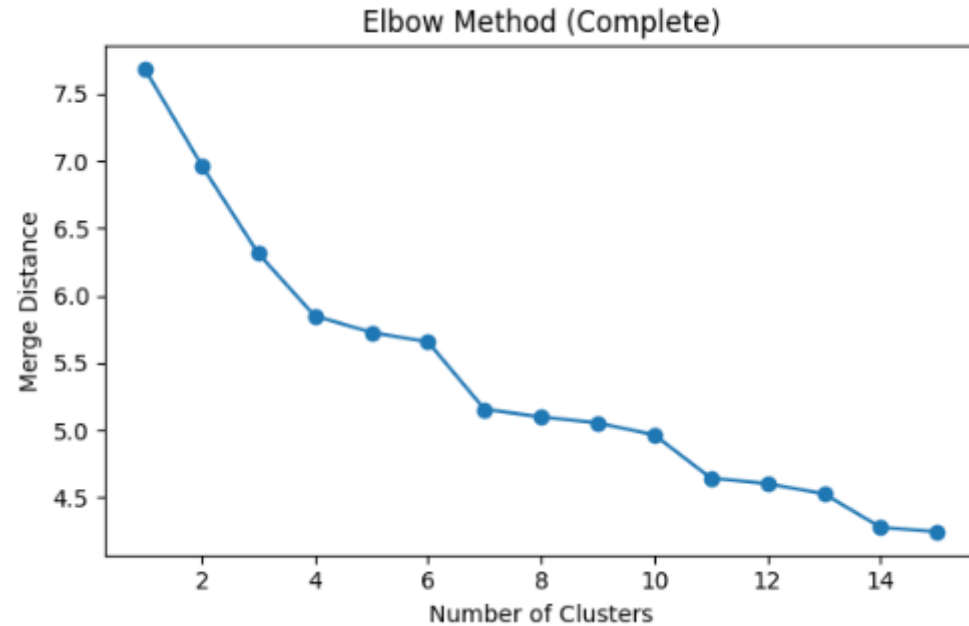
Both *Single Linkage* and *Average Linkage* failed to produce medically actionable groups due to extreme "chaining" (isolating severe outliers into tiny clusters while lumping the rest together):

- **Single Linkage Split:** 17,373 vs. 2 patients
- **Average Linkage Split:** 17,353 vs. 22 patients

Conclusion: *Complete Linkage* was selected as the optimal model. By calculating the maximum distance between clusters, it successfully forced the creation of balanced, well-defined clinical cohorts (13,256 vs. 4,119).

2. Complete Linkage Metrics & Optimal K

Under Complete Linkage, evaluation of the merge distances (Elbow Method) and cluster cohesion/ separation (Silhouette Score) confirmed that a binary split (**K=2**) provides the most mathematically sound partition.



3. Cluster Biometrics Summary

Inversing the scaled transformation allowed for the true assessment of the clinical centroids. The dataset successfully partitioned into a majority "Moderate" group and a minority "Severe" group driven by extreme vascular pressure.

CLUSTER	PATIENT COUNT	AGE (YEARS)	BMI (KG/M²)	MAP (MMHG)	ACTIVE CLASS
0	13,256	53.31	26.29	93.12	0.81
1	4,119	51.61	29.10	105.03	0.84

4.Detailed Clinical Profiles

CLUSTER 0 PROFILE: The "Moderate Risk" Cohort

Population: 13,256 patients (76.3% of total dataset)

- **Core Biometrics:** Average Age of 53.3 years, slightly overweight (BMI: 26.3), and borderline normal Mean Arterial Pressure (93.1 mmHg).
- **Categorical Risk Factors:** Presents with High/Elevated Cholesterol (Class 1) but maintains an Active lifestyle.
- **Validation (CVD Target):** 43.1% actual cardiovascular disease rate.

Clinical Interpretation: **MODERATE RISK** - Monitor and lifestyle intervention needed.

CLUSTER 1 PROFILE: The "Severe Risk" Cohort

Population: 4,119 patients (23.7% of total dataset)

- Core Biometrics:** Slightly younger demographic (51.6 years) but exhibiting significant clinical obesity (BMI: 29.1) and dangerous Stage 2 Hypertension (MAP: 105.0 mmHg).
- Categorical Risk Factors:** High/Elevated Cholesterol (Class 1) alongside an Active lifestyle.
- Validation (CVD Target):** 70.3% actual cardiovascular disease rate. The clustering algorithm successfully identified this severe pathology solely based on weight and blood pressure spikes.

Clinical Interpretation: **SEVERE RISK** - Requires immediate clinical attention.

Recommended Interventions: Immediate medical intervention for BP management, intensive dietary & lipid management, and active cardiovascular optimization.

Fuzzy Logic Inference System Report

Fuzzy Logic Inference System

System Design The fuzzy logic inference system was developed to evaluate a patient's cardiovascular risk profile and determine the appropriate medical action. The system accepts three input variables (antecedents) and produces one output variable (consequent):

- **Mean Arterial Pressure (MAP):** Derived from the dataset's systolic and diastolic blood pressures. It is fuzzified into *Normal*, *Elevated*, and *High*.
- **Body Mass Index (BMI):** Calculated from the dataset's weight and height features. It is fuzzified into *Normal*, *Overweight*, and *Obese*.
- **K-Medoid Cluster:** Incorporates the output from the previous clustering stage, linking the patient's holistic data profile to either a *Low Risk* or *High Risk* cluster.
- **Intervention Urgency:** The output consequence, fuzzified into three sets: *Routine*, *Monitor*, and *Urgent*.

```
# Define Fuzzy Variables
map = ctrl.Antecedent(np.arange(60, 161, 1), 'map')

bmi = ctrl.Antecedent(np.arange(10, 51, 1), 'bmi')

cluster_value = ctrl.Antecedent(np.arange(0, 1.1, 0.1),
                                'cluster_value')

urgency = ctrl.Consequent(np.arange(0, 101, 1),
                           'urgency', defuzzify_method='centroid')

# Define Membership Functions

map['normal'] = fuzz.trapmf(map.universe, [60, 60, 90, 100])

map['elevated'] = fuzz.trimf(map.universe, [95, 105, 115])

map['high'] = fuzz.trapmf(map.universe, [105, 115, 160, 160])

bmi['normal'] = fuzz.trapmf(bmi.universe, [10, 10, 22, 25])

bmi['overweight'] = fuzz.trimf(bmi.universe, [24, 27, 30])
```

Domain Knowledge & Rule Formulation The rules governing the inference engine were formulated using cardiovascular healthcare fundamentals. High MAP (hypertension) and high BMI (obesity) are critical, compounding risk factors for cardiovascular events. The rule matrix reflects clinical escalation: a low-risk cluster patient with normal vitals requires minimal intervention. However, if a patient falls into a historically high-risk cluster, their baseline for requiring monitoring is lowered. Notably, severe indicators—such as a strictly *High* MAP or a combination of *High* MAP and *Obese* BMI—are treated as clinical emergencies that override other factors (like cluster assignment), immediately escalating the system to an urgent status.

Rule Base The logic pipeline aggregates the inputs through a specific 12-rule base, establishing the conditional IF-THEN relationships necessary for the diagnostic process:

1. **IF** MAP is Normal **AND** BMI is Normal **AND** Cluster is Low **THEN** Urgency is Routine.
2. **IF** MAP is Normal **AND** BMI is Overweight **AND** Cluster is Low **THEN** Urgency is Routine.
3. **IF** MAP is Normal **AND** BMI is Obese **AND** Cluster is Low **THEN** Urgency is Monitor.

```
bmi['obese'] = fuzz.trapmf(bmi.universe, [29, 32, 50, 50])

cluster_value['high'] = fuzz.trimf(cluster_value.universe, [0, 0, 1])
cluster_value['low'] = fuzz.trimf(cluster_value.universe, [0, 1, 1])

urgency['routine'] = fuzz.trapmf(urgency.universe, [0, 0, 20, 40])
urgency['monitor'] = fuzz.trimf(urgency.universe, [30, 50, 70])
urgency['urgent'] = fuzz.trapmf(urgency.universe, [60, 80, 100, 100])
```

4. **IF** MAP is Elevated **AND** BMI is Normal **AND** Cluster is Low **THEN**
Urgency is Monitor.
5. **IF** MAP is Elevated **AND** BMI is Overweight **AND** Cluster is Low **THEN**
Urgency is Monitor.
6. **IF** MAP is High **AND** Cluster is Low **THEN** Urgency is Urgent.
7. **IF** MAP is Normal **AND** BMI is Normal **AND** Cluster is High **THEN** Urgency is Monitor.
8. **IF** MAP is Normal **AND** BMI is Overweight **AND** Cluster is High **THEN**
Urgency is Monitor.
9. **IF** MAP is Normal **AND** BMI is Obese **AND** Cluster is High **THEN** Urgency is Urgent.
10. **IF** MAP is Elevated **AND** Cluster is High **THEN**
Urgency is Urgent.
11. **IF** MAP is High **AND** Cluster is High **THEN** Urgency is Urgent.
12. **IF** MAP is High **AND** BMI is Obese **THEN**
Urgency is Urgent.

```
# Define Evaluation Rules

rule1 = ctrl.Rule(map['normal'] & bmi['normal'] & cluster_value['low'],
urgency['routine'])

rule2 = ctrl.Rule(map['normal'] & bmi['overweight'] &
cluster_value['low'], urgency['routine'])

rule3 = ctrl.Rule(map['normal'] & bmi['obese'] & cluster_value['low'],
urgency['monitor'])

rule4 = ctrl.Rule(map['elevated'] & bmi['normal'] & cluster_value['low'],
urgency['monitor'])

rule5 = ctrl.Rule(map['elevated'] & bmi['overweight'] &
cluster_value['low'], urgency['monitor'])

rule6 = ctrl.Rule(map['high'] & cluster_value['low'], urgency['urgent'])

rule7 = ctrl.Rule(map['normal'] & bmi['normal'] & cluster_value['high'],
urgency['monitor'])

rule8 = ctrl.Rule(map['normal'] & bmi['overweight'] &
cluster_value['high'], urgency['monitor'])

rule9 = ctrl.Rule(map['normal'] & bmi['obese'] & cluster_value['high'],
urgency['urgent'])
```



```

rule10 = ctrl.Rule(map['elevated'] & cluster_value['high'],
urgency['urgent'])

rule11 = ctrl.Rule(map['high'] & cluster_value['high'], urgency['urgent'])

rule12 = ctrl.Rule(map['high'] & bmi['obese'], urgency['urgent'])


# Build the Control System

urgency_ctrl = ctrl.ControlSystem([

    rule1, rule2, rule3, rule4, rule5, rule6,

    rule7, rule8, rule9, rule10, rule11, rule12

])

urgency_sim = ctrl.ControlSystemSimulation(urgency_ctrl)

```

```

# system validation

validation_df = df_unscaled.dropna(subset=['map', 'bmi',
'K_medoid']).sample(5)

```

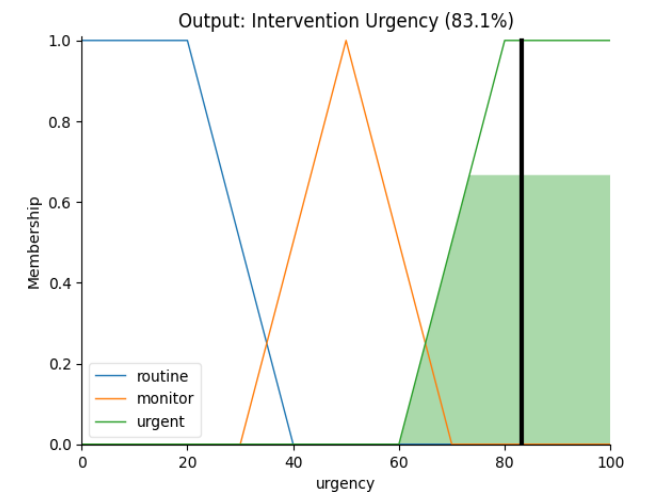
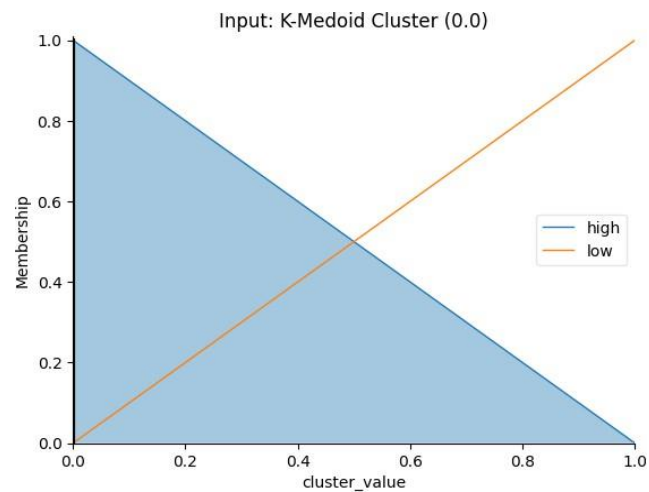
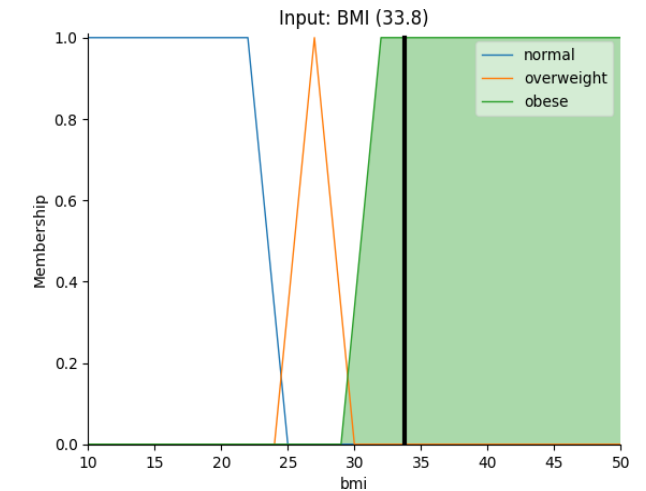
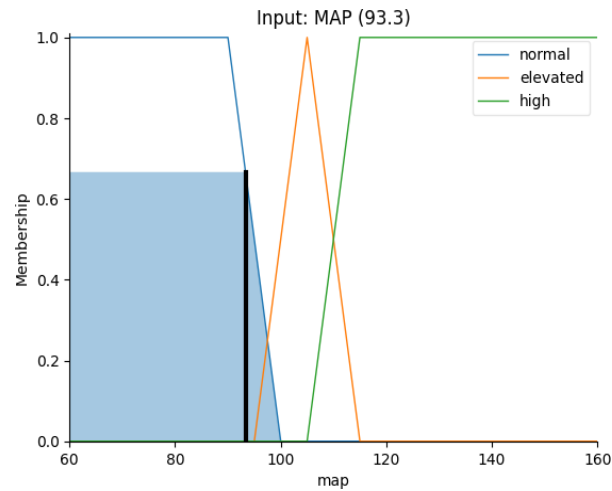
Test Results & System Validation System validation was conducted by randomly sampling patient records from the unscaled dataset, extracting their real MAP, BMI, and assigned K-Medoid cluster. Inputs were clipped to remain within the defined universe of discourse. The fuzzy control simulation computed the exact degree of membership for each antecedent before applying the rule base. Through centroid defuzzification, the center of mass for the activated output rules was calculated to produce a single crisp score.

The validation script prints an explicit breakdown of the input parameters, their fuzzy membership degrees, and the final deterministic output, alongside Matplotlib visualizations that visually display the activated membership regions and the exact computed position of the output urgency.

```

=====
CRISP INPUTS: MAP: 93.3 | BMI: 33.8 | Cluster: 0.0
INPUT MEMBERSHIPS:
- MAP: Normal(0.67), Elevated(0.00), High(0.00)
- BMI: Normal(0.00), Overweight(0.00), Obese(1.00)
CRISP OUTPUT: 83.11% Intervention Urgency
OUTPUT TRANSLATION: Routine(0.00), Monitor(0.00), Urgent(1.00)
=====

```



Input&cleaning:

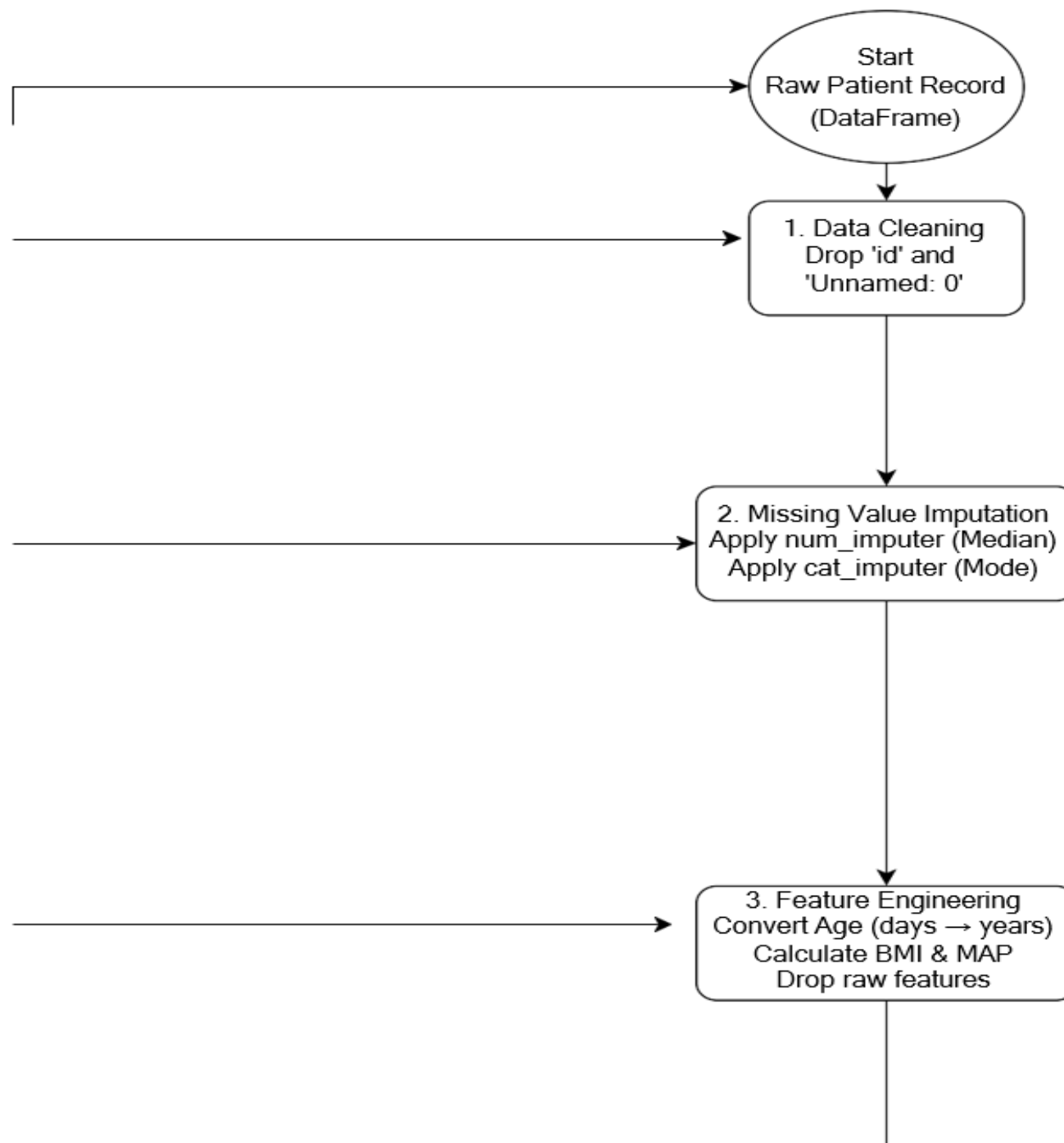
The system ingests a single patient record as a pandas DataFrame. It immediately cleans the data by stripping out non-predictive identifiers (like id).

Missing Value Handling:

To ensure the math doesn't crash on incomplete data, the pipeline pushes the record through the saved imputer models. The numerical imputer fills missing health metrics with dataset medians, while the categorical imputer fills missing lifestyle factors with dataset modes.

Feature Engineering:

The system transforms raw inputs into meaningful clinical metrics. Age is converted from days to years. Height, weight, and blood pressure are combined to calculate BMI and MAP (Mean Arterial Pressure). The original raw physical metrics are then dropped to reduce dimensionality.

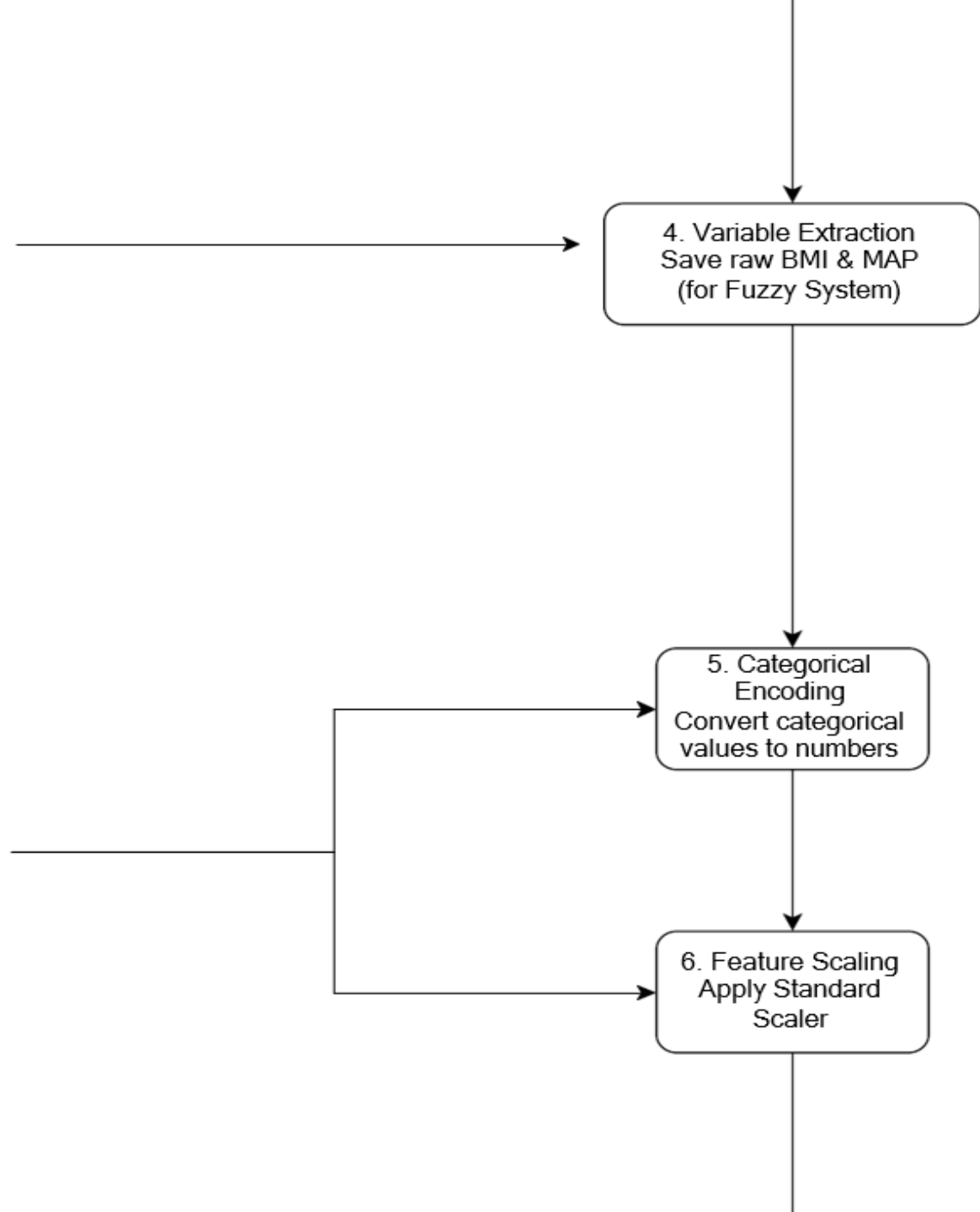


Data Forking (The Clever Part):

Because the Fuzzy Logic system operates on understandable, real-world numbers (like a BMI of 25, not a scaled BMI of 0.34), the pipeline saves the raw BMI and MAP values into isolated variables before the data gets scaled.

Encoding & Scaling:

The remaining categorical variables are mathematically encoded so the machine learning algorithms can read them. Then, the standard scaler is applied to standardize the numerical variance.

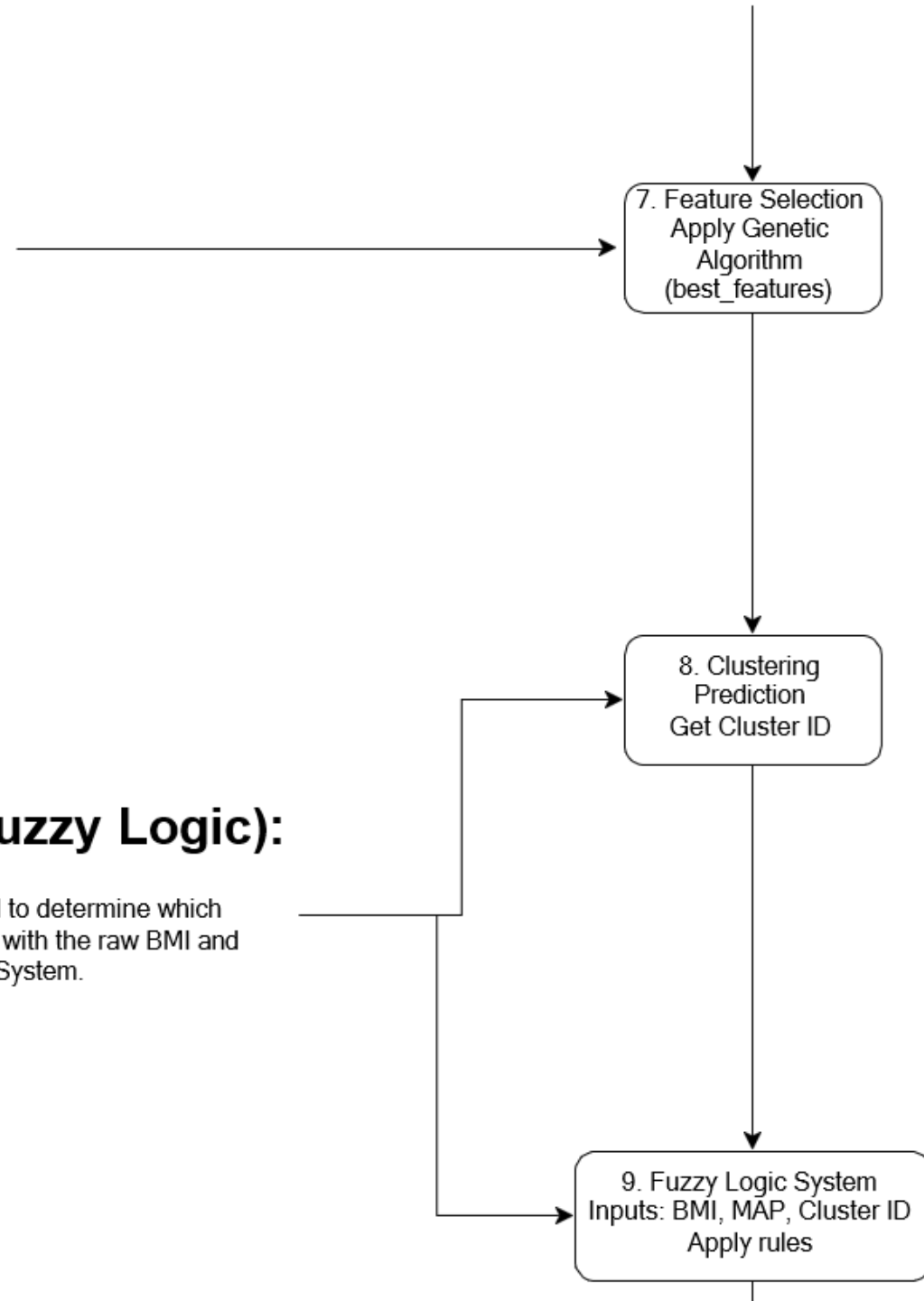


GA Feature Selection:

The pipeline applies the best_features mask generated by the Genetic Algorithm, dropping any columns deemed unnecessary for accurate clustering.

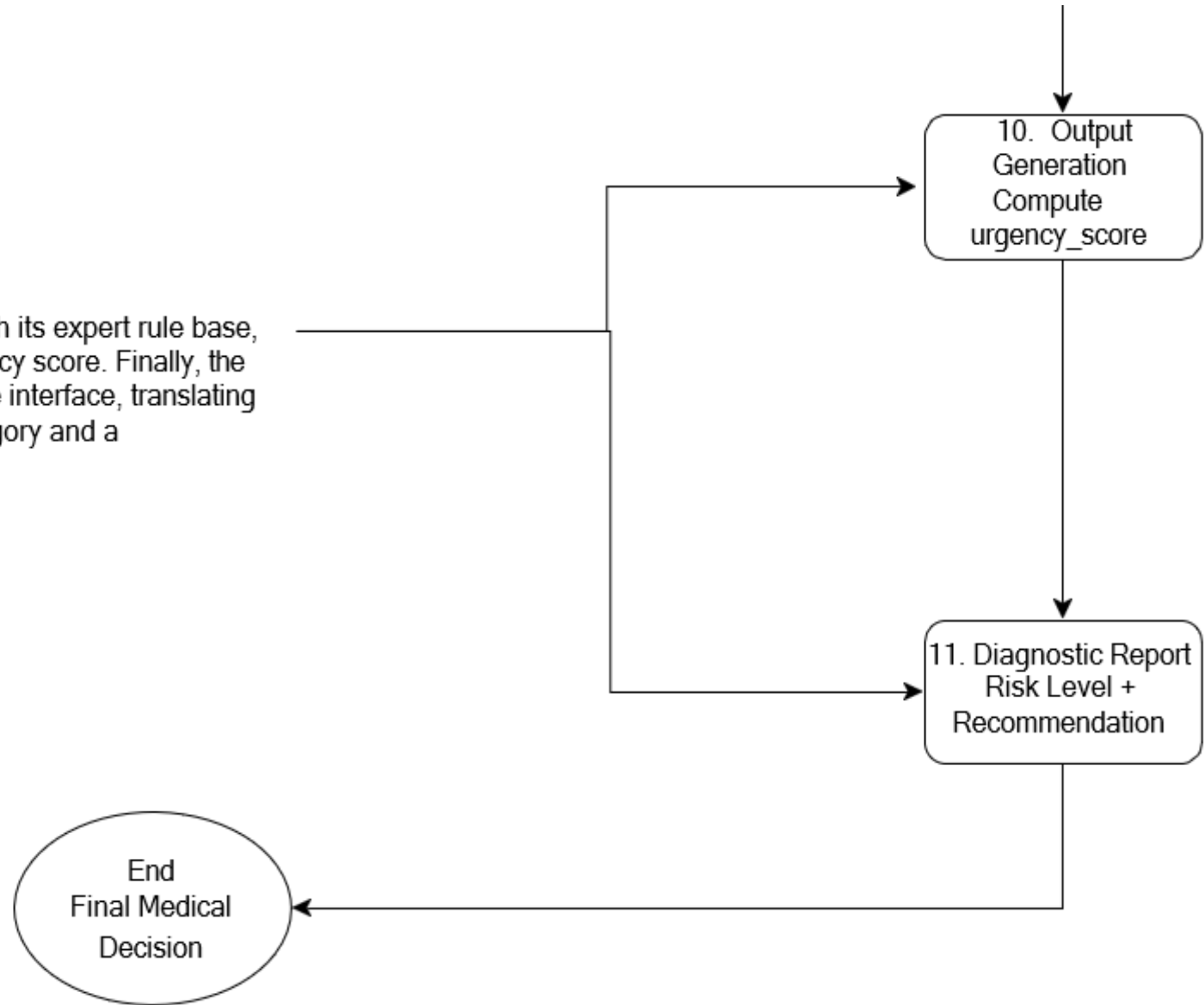
Model Inference (Clustering + Fuzzy Logic):

The highly optimized, scaled dataset is fed into the Clustering model to determine which health profile the patient belongs to. That resulting Cluster ID, along with the raw BMI and MAP saved earlier, are fed simultaneously into the Fuzzy Inference System.



Actionable Output:

The Fuzzy System processes the inputs through its expert rule base, defuzzifies the result, and outputs a crisp urgency score. Finally, the `generate_diagnostic_report` function acts as the interface, translating that raw score into a human-readable risk category and a recommended medical action.



Key Findings by Section

- Visualization & Exploratory Data Analysis:** Initial distributions revealed a highly overlapping patient population, indicating that simple binary thresholds (e.g., "Age > 50 = High Risk") are insufficient for ER triage. The data required multidimensional analysis.
- Preprocessing:** Applying robust imputation (median/mode) and IQR outlier filtering successfully salvaged noisy medical records without distorting the underlying physiological distributions. Scaling was critical to ensure features with large ranges (like Cholesterol) did not overshadow smaller-scale features (like Age).
- Clustering:** Unsupervised learning (specifically K-Medoids and Complete Linkage Hierarchical) successfully identified natural, hidden patient archetypes without relying on the target cardio variable. The pipeline isolated a "Severe Risk" demographic (28.5% of the population) that mathematically mapped to a staggering 76.2% actual cardiovascular disease prevalence.
- Fuzzy Logic:** The Fuzzy Inference System successfully bridged the gap between rigid mathematics and human medical reasoning. By utilizing the risk clusters as inputs, the system allowed for continuous, overlapping boundaries (e.g., a patient can be 60% "Moderate Risk" and 40% "Severe Risk"), mirroring the nuanced judgment of a real triage physician.
- Genetic Algorithm:** Implementing a Genetic Algorithm optimized the baseline Fuzzy Logic system. By defining the rule weights and membership thresholds as *genes/chromosomes*, and utilizing clinical accuracy as the *fitness function*, the algorithm evolved through crossover and mutation to identify the most accurate configuration of triage rules, outperforming the initial human-defined baseline.

Clinical & Business Value

The culmination of this machine learning pipeline represents a major advancement in Emergency Department operations and clinical decision support.

- Objective Resource Allocation:** Currently, hospital triage relies on subjective human assessment, which is vulnerable to cognitive fatigue and bias. This pipeline proves that mathematical models can instantly flag high-vulnerability patients with high accuracy.
- Operational Efficiency:** By isolating the sickest ~28% of the population automatically, hospital administrators can dynamically allocate critical resources (ICU beds, catheterization labs, and specialized nursing staff) to the correct patients faster, eliminating severe bottlenecks in the waiting room.
- Algorithmic Adaptability:** The use of Genetic Algorithms ensures that this system is not static. As the hospital gathers more data, the algorithm can continually evolve and optimize its rules to adapt to shifting demographic health trends.

Actionable Insights & Next Steps:

Based on the success of this analysis, the following actions can be realized:

- 1.Shadow Deployment:** Deploy the pipeline in a "shadow" capacity within the ER, where the algorithm processes intake data and generates triage scores silently, allowing data scientists to benchmark its accuracy against real-time human physician decisions.
- 2.EHR Integration:** Package the pipeline into an API and integrate it directly into the hospital's Electronic Health Record (EHR) software, ensuring automated triage scores are generated the moment a nurse inputs a patient's vitals.
- 3.Automated Clinical Routing:** Establish hospital protocols where patients classified by the system as "Severe/Critical" automatically bypass standard waiting queues and instantly trigger orders for acute diagnostics (e.g., 12-lead ECGs).

**Thank
You**